

LMSGI04 - Contenido

Documentos para el intercambio de información.



Caso práctico



Q [Nate Steiner - Flickr.com](https://www.flickr.com/photos/nate/2575639635/in/photostream/#/)

(<http://www.flickr.com/photos/nate/2575639635/in/photostream/#/>)

(CC BY)

María había salido a dar un paseo por el puerto, como todos los domingos por la mañana iba acompañada de su marido José Ramón y su hijo.

Mientras les veía jugar, estaba pensando que al día siguiente tenía que hablar con **Juan**, el técnico que se encargaba de resolver los problemas de informática de la empresa de la cual es socia. Había estado pensando si existiría algún modo de poder garantizar que

la estructura de datos de cada uno de los documentos XML que comparte con **Félix**, su socio, es la que tiene que ser y no otra, además de asegurar que todos los documentos del mismo tipo mantienen la misma estructura.

Después de haber visto la estructura general de un documento de lenguajes de marcas y estudiar los lenguajes de publicación web, nos vamos a centrar en el estudio de la construcción de vocabularios en XML.

Para ello nos ocuparemos de dos lenguajes el de DTD y el de XMLSchema.



Q  [Ministerio de Educación, Formación Profesional y Deportes \(Dominio público\)](#)

Materiales formativos de FP Online propiedad del Ministerio de Educación, Formación Profesional y Deportes.

1.- Documento. Estructura y sintaxis.



Caso práctico



Al día siguiente, cuando habla con **Juan**, sus dudas quedan disipadas. Resulta que hay varias posibilidades para asegurar una normalización en el formato de los documentos XML.

Juan comienza por describir la estructura de un documento XML y **Félix** y **María** descubren que puede ser un poco más compleja que la que habían estado usando hasta entonces para generar sus documentos.

Q [Nate Steiner](#)

(<https://www.flickr.com/photos/nate/2703328849/>)

(CC BY)

Hasta ahora hemos trabajado con documentos básicos de XML. En la primera unidad vimos que un documento XML básico estaba formado por un prólogo y un ejemplar. Recordamos que cada una de esas partes tiene el siguiente cometido:

- **Prólogo:** Informa al intérprete encargado de procesar el documento de todos aquellos datos que necesita para realizar su trabajo. Consta de dos partes:
 - **Definición de XML:** Donde se indica la versión de XML que se utiliza, el código de los datos a procesar y la autonomía del documento. Este último dato hasta ahora siempre ha sido "yes" ya que los documentos generados eran independientes.
 - **Declaración del tipo de documento:** Hasta el momento solo hemos dicho que es el nombre del ejemplar precedido de la cadena `<!DOCTYPE` y separado de ésta por, al menos un espacio.
- **Ejemplar:** Contiene los datos del documento que se quiere procesar. **Es el elemento raíz del documento y ha de ser único.** Está compuesto de elementos estructurados según una estructura de árbol en la que el elemento raíz es el ejemplar y las hojas los elementos terminales, es decir, aquellos que no contienen elementos. Los elementos pueden estar a su vez formados por atributos.

Estos documentos básicos están **incompletos**, ya que solo hemos declarado el tipo de documento que va a ser (qué ejemplar vamos a definir), pero **no hemos definido qué cualidades tiene ese ejemplar**.

Para completarlos y definir en profundidad las cualidades del ejemplar, usaremos o bien DTDs o bien XML Schemas.



Autoevaluación

Marcar los componentes de un documento XML:

- ☐ Prólogo.
- ☐ Ejemplar.
- ☐ Definición de codificación del documento.
- ☐ Cabecera.

Mostrar retroalimentación

Solución

1. Correcto (#answer-44_139)
 2. Correcto (#answer-44_269)
 3. Correcto (#answer-44_271)
 4. Incorrecto (#answer-44_273)
-

1.1.- Declaración de tipo de documento.



Q Petra Gregorová - www.flickr.com (

CC BY-ND)

Ya habíamos visto que permite **al autor definir restricciones y características en el documento**, aunque no habíamos profundizado en las partes que la forman:

La declaración del tipo de documento propiamente dicha. Comienza con el texto que indica el nombre del tipo, precedido por la cadena " " separado del nombre del tipo por, al menos, un espacio. El nombre del tipo ha de ser idéntico al del ejemplar del documento **XML**, en el que se está trabajando.

- **La definición del tipo de documento.** Permite asociar al documento una definición de tipo **DTD**, la cual se encarga de definir las cualidades del tipo. Es decir, define los tipos de los elementos, atributos y notaciones que se pueden utilizar en el documento así como las restricciones del documento, valores por defecto, etc. Para formalizar todo esto, XML está provisto de ciertas estructuras llamadas **declaraciones de marcado**, las cuales pueden ser internas o externas. Normalmente un documento XML se compone de una mezcla de declaraciones de marcado internas y externas. En este último caso debe expresarse en el documento dónde encontrar las declaraciones, así como indicar en la declaración de XML que el documento no es autónomo. Las diferencias entre estos tipos de declaraciones de marcado dan lugar a dos subconjuntos el interno y el externo, conviene saber que primero se procesa el subconjunto interno y después el externo, lo que permite sobrescribir declaraciones externas compartidas entre varios documentos y ajustar el DTD a un documento específico.

- **Subconjunto interno:** Contiene las **declaraciones que pertenecen exclusivamente a un documento** y no es posible compartirlas. Se localizan dentro de unos corchetes que siguen a la declaración de tipo del documento.
- **Subconjunto externo:** Están localizadas en un documento con extensión **dtd** que puede situarse en el mismo directorio que el documento XML. Habitualmente son declaraciones que pueden ser compartidas entre múltiples documentos XML que pertenecen al mismo tipo. En este caso la declaración de documento autónomo ha de ser negativa, ya que es necesario el fichero del subconjunto externo para la correcta interpretación del documento. Con ello el procesado del documento será

más lento, ya que antes de procesar el documento el procesador ha de obtener todas las entidades.

- `<!DOCTYPE nombre_ejemplar SYSTEM "URI">`
En este caso, se especifica un URI donde pueden localizarse las declaraciones.
- `<!DOCTYPE nombre_ejemplar PUBLIC "id_publico" "URI">`
En este caso también se especifica un identificador, que puede ser utilizado por el procesador XML para intentar generar un URI alternativo, posiblemente basado en alguna tabla. **Como se puede observar también es necesario incluir algún URI.**



Autoevaluación

La definición de tipo de documento ha de ser:

- ☐ Interna o externa al documento XML al que está referida.
- ☐ Interna al documento XML que le refiere.
- ☐ Externa al documento XML que le refiere.
- ☐ Única.

Muy bien. Has captado la idea. Correcta.

Incorrecta, porque puede ser externa.

No es la respuesta correcta porque puede definirse en el documento.

No es correcta ya que un documento puede tener una definición DTD interna u otra externa definidas simultáneamente.

Solución

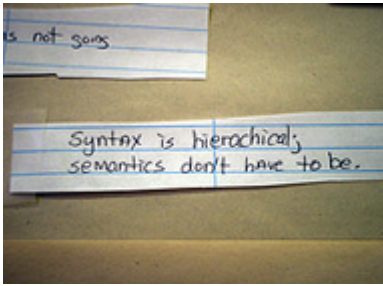
1. Opción correcta (#answer-45_144)

2. Incorrecto (#answer-45_275)

3. Incorrecto (#answer-45_278)

4. Incorrecto (#answer-45_281)

1.2.- Definición de la sintaxis de documentos XML.



Q Quinn.annya - www.flickr.com (
CC BY-SA)

Recordamos que en estos documentos las etiquetas de marcado describen la estructura del documento.

Un elemento es un grupo formado por una etiqueta de apertura, otra de cierre y el contenido que hay entre ambas.

En los documentos de lenguajes de marcas, la distribución de los elementos está jerarquizada según una estructura de árbol, lo que implica que es posible anidarlos pero no entrelazarlos.

Hemos visto que en los elementos el orden es importante, ¿lo es también para los atributos? En este caso el orden no es significativo. Lo que hay que tener presente es que no puede haber dos atributos con el mismo nombre.

Sabemos que los atributos no pueden tener nodos que dependan de ellos, por tanto solo pueden corresponder con hojas de la estructura de árbol que jerarquiza los datos. ¿Significa esto que todas las hojas van a ser atributos? Pues no, es cierto que los atributos son hojas, pero las hojas pueden ser atributos o elementos.

En ese caso, ¿qué criterios podemos utilizar para decidir si un dato del documento que se pretende estructurar ha de representarse mediante un elemento o un atributo? Aunque no siempre se respetan, podemos usar los siguientes criterios:

- **El dato será un elemento si cumple alguna de las siguientes condiciones:**
 - Contiene subestructuras.
 - Es de un tamaño considerable.
 - Su valor cambia frecuentemente.
 - Su valor va a ser mostrado a un usuario o aplicación.
- **Los casos en los que el dato será un atributo son:**
 - El dato es de pequeño tamaño y su valor raramente cambia, aunque hay situaciones en las que este caso puede ser un elemento.
 - El dato solo puede tener unos cuantos valores fijos.
 - El dato guía el procesamiento XML, pero no se va a mostrar.

Los espacios de nombres, o namespaces, ¿qué nos permiten?

- Diferenciar entre los elementos y atributos de distintos vocabularios con diferentes significados que comparten nombre.
- Agrupar todos los elementos y atributos relacionados de una aplicación XML para que el software pueda reconocerlos con facilidad.

¿Cómo se declaran?

`xmlns:"URI_namespace"`

¿Y si se usa un prefijo que nos informe sobre cuál es el vocabulario al que está asociada esa definición?

`xmlns:prefijo="URI_namespace"`

En ambos casos `URI_namespace` es la localización del conjunto del vocabulario del espacio de nombres al que se hace referencia.



Autoevaluación

Marcar las afirmaciones válidas que hacen referencia a un espacio de nombres.

- ☐ Facilitan que el software localice el vocabulario de la aplicación.
- ☐ Define los atributos que forman parte de un documento.
- ☐ Permiten tener varios elementos homónimos con diferentes significados.
- ☐ Determinan los elementos que forman parte de un documento XML.

Mostrar retroalimentación

Solución

1. Correcto (#answer-46_139)
 2. Incorrecto (#answer-46_284)
 3. Correcto (#answer-46_286)
 4. Incorrecto (#answer-46_288)
-

2.- Definiciones de tipo de documento, DTD.



Caso práctico



Según **Juan** el método más sencillo para intentar normalizar los documentos con los que trabajan, consiste en definir unos vocabularios que han de cumplir los documentos que generan, estos se llaman Definición de Tipo de Documento. Además, aunque no es un lenguaje **XML**, tiene una sintaxis sencilla y fácil para que ella y **Félix** puedan comprenderla y utilizarla.

Q [Jonny Goldstein](#)

(<https://www.flickr.com/photos/jonnygoldstein/471761750/>)

(CC BY)

Están formadas por una **relación precisa de qué elementos pueden aparecer en un documento y dónde, así como el contenido y los atributos del mismo. Garantizan que los datos del documento XML cumplen las restricciones que se les haya impuesto en el DTD**, ya que estas últimas permiten:

- ☐ Especificar la estructura del documento.
- ☐ Reflejar una [restricción de integridad referencial](#)  mínima utilizando (**ID** e **IDREF**).
- ☐ Utilizar unos pequeños mecanismos de abstracción comparables a las [macros](#) , que son las entidades.
- ☐ Incluir documentos externos.

¿Cuáles son los inconvenientes de los DTD?

Los principales son:

- ☐ Su sintaxis no es XML.
- ☐ No soportan espacios de nombres.
- ☐ No definen tipos para los datos. Solo hay un tipo de elementos terminales, que son los datos textuales.
- ☐ No permite las secuencias no ordenadas.
- ☐ No es posible formar claves a partir de varios atributos o elementos.
- ☐ Una vez que se define un DTD no es posible añadir nuevos vocabularios.

Cuando están definidas dentro del documento XML se ubican entre corchetes después del nombre del ejemplar en el elemento **<!DOCTYPE>** pero, cuando

está definido en un fichero externo ¿a qué tipo de fichero corresponde? Definimos el DTD externo en un fichero de texto plano con extensión **dtd**.



Autoevaluación

Marcar las afirmaciones referidas a un DTD:

- ☐ Permite la formación de identificadores compuestos por varios atributos.
- ☐ Para construirlas no se usa un lenguaje basado en XML.
- ☐ Permite incluir ficheros binarios en el documento.
- ☐ Definen distintos tipos para los datos.

Mostrar retroalimentación

Solución

1. Incorrecto (#answer-48_139)
 2. Correcto (#answer-48_290)
 3. Correcto (#answer-48_292)
 4. Incorrecto (#answer-48_294)
-

2.1.- Declaraciones de tipos de elementos terminales.



Los **tipos terminales** son aquellos elementos que se corresponden con hojas de la estructura de árbol formada por los datos del documento **XML** asociado al **DTD**. La declaración de tipos de elementos está formada por la cadena `<!ELEMENT>` separada por, al menos un espacio del nombre del elemento **XML** que se declara, y seguido de la declaración del contenido que puede tener dicho elemento.

Q [quapan - www.flickr.com](http://www.flickr.com/photos/quapan/) (
CC BY)

En el caso de elementos terminales, es decir, aquellos que no contienen más elementos, esta declaración de contenido es dada por uno de los siguientes valores:

- **EMPTY**: Indica que el elemento no es contenedor. Por ejemplo, la siguiente definición muestra un elemento A que no contiene nada:

```
<!ELEMENT A EMPTY>
```

- **ANY**: Permite que el contenido del elemento sea cualquier cosa. Un ejemplo de definición de un elemento de este tipo es:

```
<!ELEMENT A ANY>
```

- **(#PCDATA)**: Indica que los datos son analizados en busca de etiquetas, resultando que el elemento no puede contener elementos, es decir solo puede contener datos de tipo carácter exceptuando los siguientes: `<`, `&`, `]]`, `>`. Si es de este tipo, el elemento A tendrá una definición como:

```
<!ELEMENT A (#PCDATA)>
```

EJEMPLO

Si se tiene la siguiente estructura en un documento **XML**

```
<alumno>Olga Velarde Cobo</alumno>
```

Un **DTD** que podría ajustarse sería:

<!ELEMENT alumno (#PCDATA)>



Autoevaluación

Los elementos terminales de tipo ANY son aquellos que están:

- ☐ Formados por cadenas de texto exclusivamente.
- ☐ Vacíos
- ☐ Formados por cualquier cosa.
- ☐ Estará formada por otros elementos.

No es correcta porque pueden estar formados por cualquier cosa.

Incorrecta, porque también pueden ser otra cosa.

Ánimo, sigue así. ¡Correcta!

No es correcta, los elementos terminales son aquellos que no están formados por otros elementos.

Solución

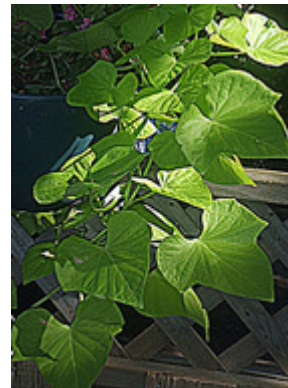
1. Incorrecto (#answer-49_144)
 2. Incorrecto (#answer-49_302)
 3. Opción correcta (#answer-49_305)
 4. Incorrecto (#answer-49_308)
-

2.2.- Declaraciones de tipos de elementos no terminales.

Una vez que sabemos el modo de definir las hojas de un árbol de datos veamos cómo definir sus ramas, es decir los elementos que están formados por otros elementos.

Para definirlos utilizamos referencias a los grupos que los componen tal y como muestra el ejemplo:

```
<!ELEMENT A (B, C)>
```



Q

En este caso se ha definido un elemento A que está formado por un elemento B seguido de un elemento C.

Bill Barber - www.flickr.com
(CC BY-NC)

¿Y qué sucede cuando un elemento puede aparecer en el documento varias veces, hay que indicarlo de algún modo? Pues sí, también hay que indicar cuando un elemento puede no aparecer. Para ello usamos los siguientes operadores, que nos permiten definir la [cardinalidad](#)  de un elemento:

- Operador opción, ?. Indica que el elemento no es obligatorio. En el siguiente ejemplo el subelemento trabajo es opcional.

```
<!ELEMENT telefono (trabajo?, casa )>
```

- Operador uno-o-más, +. Define un componente presente al menos una vez. En el ejemplo definimos un elemento formado por el nombre de una provincia y otro grupo, que puede aparecer una o varias veces.

```
<!ELEMENT provincia (nombre, (cp, ciudad)+ )>
```

- Operador cero-o-mas, *. Define un componente presente cero, una o varias veces. En el ejemplo el grupo (cp, ciudad) puede no aparecer o hacerlo varias veces.

```
<!ELEMENT provincia (nombre, (cp, ciudad)* )>
```

- Operador de elección, |. Cuando se utiliza sustituyendo las comas en la declaración de grupos indica que para formar el documento **XML** hay que elegir entre los elementos separados por este operador. En el ejemplo siguiente, el documento XML tendrá elementos provincia que estarán formados por el elemento nombre y el cp (código postal), o por el elemento nombre y la ciudad.

```
<!ELEMENT provincia (nombre, (cp | ciudad) )>
```

EJEMPLO

Si se tiene la siguiente estructura en un documento **XML**

```
1 | <alumno>
2 | <nombre>Olga</nombre>
3 | <dirección>El Percebe 13</dirección>
4 | </alumno>
```

Un **DTD** que podría ajustarse sería:

```
1 | <!ELEMENT alumno (nombre, direccion)>
2 | <!ELEMENT nombre (#PCDATA)>
3 | <!ELEMENT dirección (#PCDATA)>
```



Autoevaluación

La diferencia entre el operador ? y el + es que el primero permite que el elemento sobre el que se aplica esté presente una vez, como máximo, mientras que el operador + no limita el número máximo de veces que está presente el elemento en el documento XML.

☐ Verdadero.

☐ Falso.

Muy bien, correcto.

Incorrecto, el operador ? es el que determina que un elemento es opcional mientras que + determina que un elemento aparecerá una o más veces.

Solución

1. Opción correcta (#answer-50_144)

2. Incorrecto (#answer-50_311)

2.3.- Declaraciones de listas de atributos para los tipos de elementos.

Ya sabemos cómo declarar elementos, ahora veamos el modo de declarar los atributos asociados a un elemento. Para ello utilizamos la cadena `<!ATTLIST` seguida del nombre del elemento asociado al atributo que se declara, luego el nombre de éste último seguido del tipo de atributo y del modificador. Este elemento puede usarse para declarar una lista de atributos asociada a un elemento, o repetirse el número de veces necesario para asociar a dicho elemento esa lista de atributos, pero individualmente.

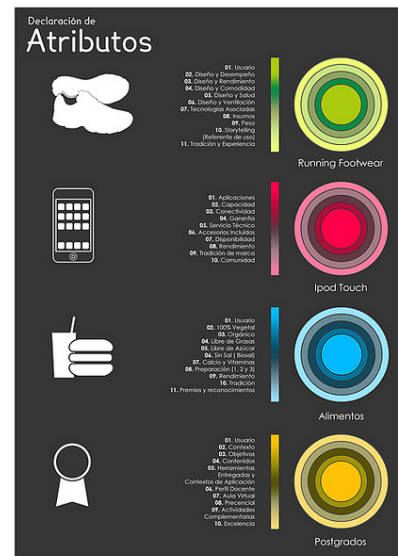
Al igual que los elementos no todos los atributos son del mismo tipo, los más destacados son:

- **CDATA** , se utiliza cuando el atributo es una cadena de texto.
- **ID** , permite declarar un atributo identificador en un elemento. Hay que recordar que este valor ha de ser único en el documento. Además hay que tener en cuenta que los números no son nombres válidos en **XML**, por tanto no son un identificador legal de XML. Para resolverlo suele incluirse un prefijo en los valores y separarlo con un guión o una letra.
- **IDREF** , permite hacer referencias a identificadores. En este caso el valor del atributo ha de corresponder con el de un identificador de un elemento existente en el documento.
- **NMTOKEN** , permite determinar que el valor de un atributo ha de ser una sola palabra compuesta por los caracteres permitidos por **XML**.

¿También hemos de declarar si el valor de un atributo es obligatorio o no? Si, para ello se usan los siguientes modificadores:

- **#IMPLIED** , determina que el atributo sobre el que se aplica es opcional.
- **#REQUIRED** , determina que el atributo tiene carácter obligatorio.
- **#FIXED** , permite definir un valor fijo para un atributo independientemente de que ese atributo se defina explícitamente en una instancia del elemento en el documento **XML**.
- **Literal**, asigna a un atributo el valor dado por una cadena entre comillas.
- **Enumeración**, es decir, el atributo solo puede tomar uno de los valores determinados dentro de un paréntesis y separados por el operador |.

```
<!ATTLIST fecha dia_semana
(lunes|martes|miércoles|jueves|viernes|sábado|domingo)
#REQUIRED>
```



fif

Q frvelizfadic - www.flickr.com (

CC BY-NC-ND)

EJEMPLO

Si se tiene la siguiente estructura en un documento **XML**

```
1  <alumno edad=15>
2  <nombre>Olga</nombre>
3  <apellidos>Velarde Cobo</apellidos>
4  <dirección>El Percebe 13</dirección>
5  </alumno>
```

Un **DTD** que podría ajustarse sería:

```
1  <!ELEMENT alumno (nombre, apellidos, direccion)>
2  <!ATTLIST alumno edad CDATA #REQUIRED>
3  <!ELEMENT nombre (#PCDATA)>
4  <!ELEMENT apellidos (#PCDATA)>
5  <!ELEMENT dirección (#PCDATA)>
```



Autoevaluación

¿Cuál de los siguientes atributos permite añadir a los datos una restricción de integridad referencial?

- ☐ MTOKEN.
- ☐ ID.
- ☐ IDREF.
- ☐ CDATA.

Incorrecto, los datos de este tipo garantizan que tienen un nombre que cumple las normas de XML.

No es correcto. Este atributo impone la restricción que ese valor debe ser único para ese atributo dentro de un documento XML.

Genial, es correcto.

No está bien. Este tipo de datos se usa para cadenas de texto.

Solución

1. Incorrecto (#answer-51_144)
 2. Incorrecto (#answer-51_314)
 3. Opción correcta (#answer-51_317)
 4. Incorrecto (#answer-51_320)
-

2.4.- Declaraciones de entidades.

¿Qué sucede si queremos declarar valores constantes dentro de los documentos? ¿podemos?

Las entidades nos permiten definir constantes en un documento **XML**. Cuando se usan dentro del documento **XML** se limitan por "&" y ";", por ejemplo `&entidad;`

¿Cómo trabaja el intérprete con ellos? Al procesar el documento **XML**, el intérprete sustituye la entidad por el valor que se le ha asociado en el **DTD**.

No admiten recursividad, es decir, una entidad no puede hacer referencia a ella misma.

Para definir una entidad en un **DTD** se usa el elemento `<!ENTITY>`

Las entidades pueden ser de cuatro tipos:

□ Internas: Existen cinco entidades predefinidas en el lenguaje, son:

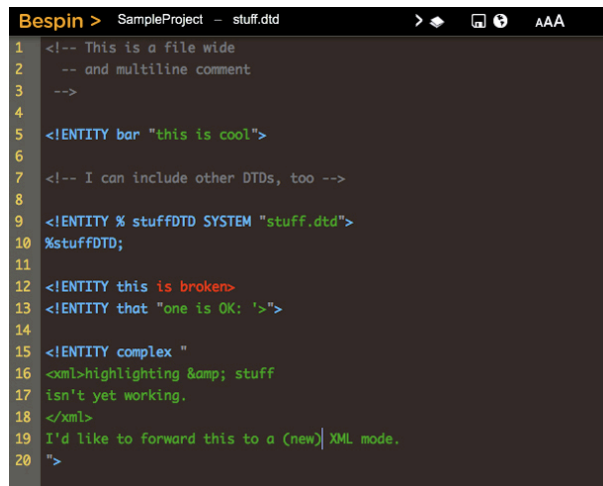
- `<`: Se corresponde con el signo menor que, <.
- `>`: Hace referencia al signo mayor que, >.
- `"`: Son las comillas rectas dobles, ".
- `'`: Es el apóstrofe o comilla simple, '.
- `&`: Es el ampersand, &.

¿Se puede definir una entidad diferente? ¿Cómo? Utilizando la siguiente sintaxis: `<!ENTITY nombre_entidad "valor de la entidad">` Por ejemplo,

```
<!ENTITY dtd "Definiciones de Tipo de Documento">
```

□ Externas: Permiten establecer una relación entre el documento **XML** y otro documento a través de la URL de éste último. Un ejemplo de declaración de una entidad externa es:

```
<!ENTITY nombre_entidad SYSTEM "http://localhost/docxml/fichero_entidad.xml">
```



The screenshot shows a code editor with the following content:

```
Bespin > SampleProject - stuff.dtd
1 <!-- This is a file wide
2 -- and multiline comment
3 -->
4
5 <!ENTITY bar "this is cool">
6
7 <!-- I can include other DTDs, too -->
8
9 <!ENTITY % stuffDTD SYSTEM "stuff.dtd">
10 %stuffDTD;
11
12 <!ENTITY this is broken>
13 <!ENTITY that "one is OK: '>'>
14
15 <!ENTITY complex "
16 <xml>highlighting &amp; stuff
17 isn't yet working.
18 </xml>
19 I'd like to forward this to a (new) XML mode.
20 ">
```

Q Axel Hecth - www.flickr.com (CC BY-ND)

En este caso el contenido de los ficheros es analizado, por lo que deben seguir la sintaxis **XML**.

Cuando es necesario incluir ficheros con formatos binarios, es decir ficheros que no se analicen, se utiliza la palabra reservada **NDATA** en la definición de la entidad y habrá que asociar a dicha entidad una declaración de notación, tal y como muestra el ejemplo del apartado siguiente.

- De parámetro: Permite dar nombres a partes de un **DTD** y hacer referencia a ellas a lo largo del mismo. Son especialmente útiles cuando varios elementos del **DTD** comparten listas de atributos o especificaciones de contenidos. Se denotan por **%entidad**;

```
1 <!ENTITY %direccion "calle, numero?, ciudad, cp">
2 <!ENTITY alumno (dni, %direccion);>
3 <!ENTITY ies (nombre, %direccion);>
```

- De parámetro externas: Permite incluir en un **DTD** elementos externos, lo que se aplica en dividir la definición **DTD** en varios documentos.

```
<!ENTITY persona SYSTEM "persona.dtd">
```



Autoevaluación

Las entidades permiten definir elementos cuyo valor es constante dentro de un DTD.

- ☐ Verdadero.
- ☐ Falso.

Entre otras cosas permiten definir valores constantes.

Genial, es correcto.

Solución

1. Incorrecto (#answer-52_144)
 2. Opción correcta (#answer-52_323)
-

2.5.- Declaraciones de notación.



Q MattieTK - www.flickr.com (
CC BY-NC-SA)

Cuando se incluyen ficheros binarios en un fichero, ¿cómo le decimos qué aplicación ha de hacerse cargo de ellos? La respuesta es utilizando notaciones. La sintaxis para declarar notaciones es:

```
<!NOTATION nombre SYSTEM aplicacion>
```

Por ejemplo, una notación llamada **gif** donde se indica que se hace referencia a un editor de formatos **gif** para visualizar imágenes será:

```
<!NOTATION gif SYSTEM "gifEditor.exe">
```

Para asociar una entidad externa no analizada a esta notación basta declarar dicha entidad del siguiente modo:

```
<!ENTITY dibujo SYSTEM "imagen.gif" NDATA gif>
```



Autoevaluación

Las notaciones permiten:

- ☐ Establecer el modo en el que se ha de estructurar un fichero **XML**.
- ☐ Determinar cuál es la aplicación que ha de procesar un fichero binario asociado a un fichero XML.
- ☐ Asociar un fichero binario a un fichero XML.
- ☐ Definir las etiquetas válidas en un fichero XML.

No es correcta pues sirve para determinar la aplicación que ha de tratar un fichero binario.

Estupendo, continúa como hasta ahora.

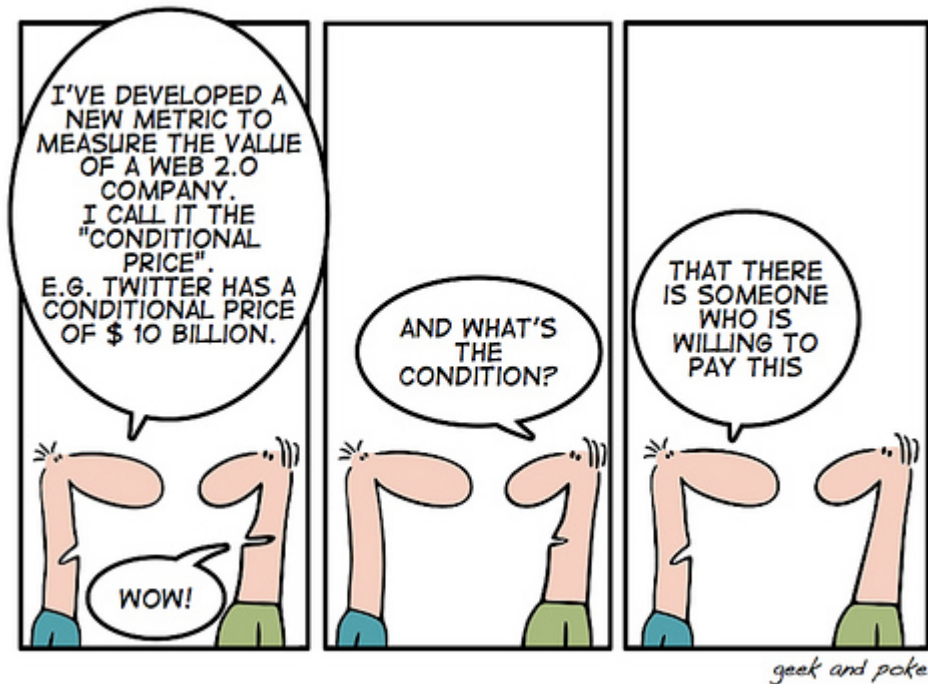
No es la respuesta correcta porque eso es lo que hace una entidad.

Incorrecto, esa es la labor del DTD.

Solución

1. Incorrecto (#answer-55_144)
 2. Opción correcta (#answer-55_326)
 3. Incorrecto (#answer-55_329)
 4. Incorrecto (#answer-55_332)
-

2.6.- Secciones condicionales.



Geek&Poke - www.flickr.com (CC BY-ND)

Permiten incluir o ignorar partes de la declaración de un **DTD**. Para ello se usan dos [tokens](#) :

- **INCLUDE** , permite que se vea esa parte de la declaración del **DTD**. Su sintaxis es:

```
<![INCLUDE [Declaraciones visibles] ] >
```

Por ejemplo:

```
<![INCLUDE [ <!ELEMENT nombre (#PCDATA)>] ] >
```

- **IGNORE** , permite ocultar esa sección de declaraciones dentro del **DTD**. La forma de uso es:

```
<![IGNORE [Declaraciones ocultas] ] >
```

Por ejemplo:

```
<![IGNORE [<!ELEMENT clave (#PCDATA)>] ] >
```



Autoevaluación

Las sentencias condicionales permiten definir unos elementos u otros dentro del fichero **XML** en función de una determinada condición.

- ☐ Verdadero.
- ☐ Falso.

Será mejor que leas de nuevo el apartado.

Muy bien. Has captado la idea...

Solución

1. Incorrecto (#answer-56_144)
 2. Opción correcta (#answer-56_335)
-

3.- Esquemas.



Caso práctico



Félix, quien considera que la normalización de los documentos **XML** que manejan en la empresa va a ser un duro trabajo para **María**, él y otros trabajadores inexpertos, plantea la posibilidad de que se encargue de ello algún trabajador de la consultoría informática que dirige **Juan**.

Q [Let Ideas Compete](#)

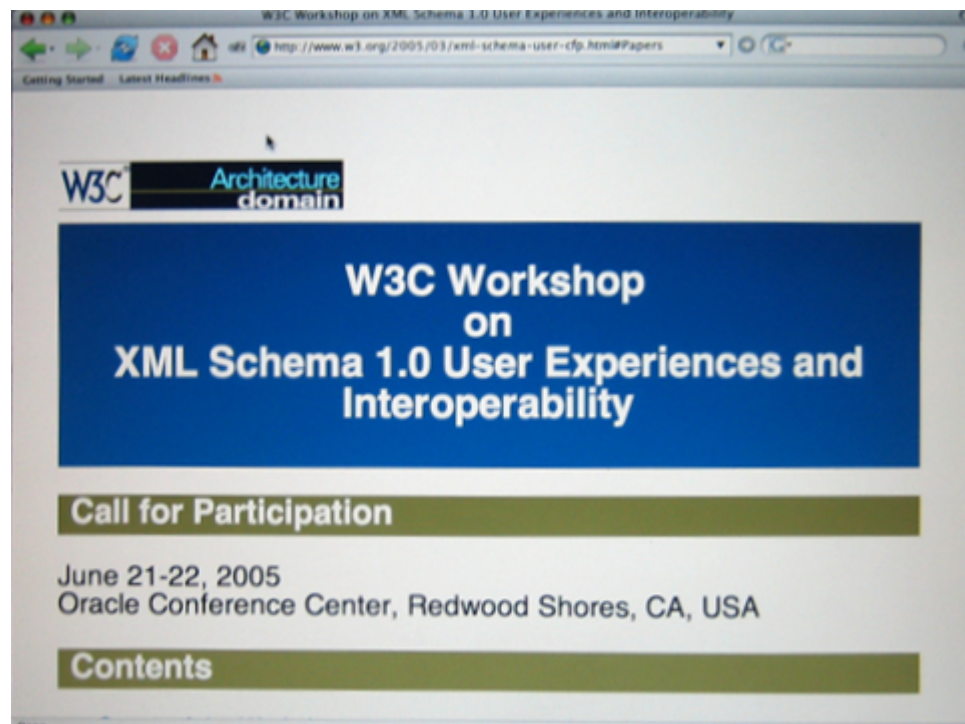
(<https://www.flickr.com/photos/question-everything/554106149/>)

(CC BY-NC-ND)

Al final se va a encargar de ello **Marina**. Les explica que, en lugar de trabajar con **DTD's** le parece mejor hacerlo con un lenguaje

XML llamado XML Schema, el cual tiene, entre otras, la ventaja de permitir definir el tipo de datos de cada uno de los componentes de cada documento.

Los
DTD



Q [psd - www.flickr.com](#) (CC BY)

permiten diseñar un vocabulario para ficheros **XML**, pero, ¿qué sucede cuando los valores de los elementos y atributos de esos ficheros han de corresponder a datos de un tipo determinado, o cumplir determinadas restricciones que no pueden reflejarse en los **DTD**? Para ello se definen **XML Schemas**.

¿También se definen en ficheros planos? Si, ya que son documentos **XML**, pero en este caso la extensión de los archivos es **xsd**, motivo por el cual también se les denomina documentos **XSD**.

Los elementos **XML** que se utilizan para generar un esquema han de pertenecer al espacio de nombre **XML Schema**, que es: <http://www.w3.org/2001/XMLSchema>.

El ejemplar de estos ficheros es `<xs:schema>`, contiene declaraciones para todos los elementos y atributos que puedan aparecer en un documento **XML** asociado válido. Los elementos hijos inmediatos de este ejemplar son `<xs:element>` que nos permiten crear globalmente un elemento. Esto significa que el elemento creado puede ser el ejemplar del documento **XML** asociado.

EJEMPLO

Vamos a crear un esquema correspondiente al siguiente documento **XML**:

```
1 <?xml version="1.0" encoding="UTF-8" standalone="yes"?>
2 <!DOCTYPE alumno>
3 <alumno>Olga Velarde Cobo</alumno>
```

Un **XML Schema** que podría ajustarse sería:

```
1 <?xml version="1.0" encoding="UTF-8" standalone="yes"?>
2 <xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
3 <xs:element name="alumno" type="xs:string"/>
4 </xs:schema>
```



Debes conocer

En este primer enlace encontrarás los fundamentos del estándar XML Schema.

[XML Schema Fundamentos](#)

3.1.- Tipos de datos.

Son los distintos valores que puede tomar el atributo `type` cuando se declara un elemento o un atributo y representan el tipo de dato que tendrá el elemento o atributo asociado a ese `type` en el documento XML.



Q floresyplantas.net (CC BY-NC-SA)

Algunos de estos valores predefinidos son:

- `string` , se corresponde con una cadena de caracteres UNICODE.
- `boolean` , representa valores lógicos, es decir que solo pueden tomar dos valores, true o false.
- `integer` , número entero positivo o negativo.
- `positiveInteger` , número entero positivo.
- `negativeInteger` , número entero negativo.
- `decimal` , número decimal, por ejemplo, 8,97.
- `dateTime` , representa una fecha y hora absolutas.
- `duration` , representa una duración de tiempo expresado en años, meses, días, horas, minutos segundos. El formato utilizado es: `PnYnMnDTnHnMnS` . Por ejemplo para representar una duración de 2 años, 4 meses, 3 días, 5 horas, 6 minutos y 10 segundos habría que poner: `P2Y4M3DT5H6M7S`. Se pueden omitir los valores nulos, luego una duración de 2 años será `P2Y`. Para indicar una duración negativa se pone un signo – precediendo a la P.
- `time` , hora en el formato `hh:mm:ss` .
- `date` , fecha en formato CCYY-MM-DD.
- `gYearMonth` , representa un mes de un año determinado mediante el formato CCYY-MM.
- `gYear` , indica un año gregoriano, el formato usado es CCYY.
- `gMonthDay` , representa un día de un mes mediante el formato –MM-DD.
- `gDay` , indica el ordinal del día del mes mediante el formato –DD, es decir el 4º día del mes será –04.
- `gMonth` , representa el mes mediante el formato –MM. Por ejemplo, febrero es –02.
- `anyURI` , representa una URI.
- `language` , representa los identificadores de lenguaje, sus valores están definidos en REC 1766.
- `ID`, `IDREF`, `ENTITY`, `NOTATION`, `MTOKEN` . Representan lo mismo que en los DTD



Debes conocer

En este enlace encontrarás los tipos de datos admitidos por el estándar.

<https://www.w3.org/TR/xmlschema-2/>



Autoevaluación

¿Cuál de los siguientes tipos no hace referencia a un dato de tiempo?

- ☐ dateTime.
- ☐ duration.
- ☐ gDayMonth.
- ☐ gMonthDay.

Incorrecta, porque la duración es una medida de tiempo.

Incorrecta, porque la duración es una medida de tiempo.

Correcta, muy bien.

No es correcta ya que este tipo corresponde a un día de un mes de un año cualquiera.

Solución

1. Incorrecto (#answer-59_144)
 2. Incorrecto (#answer-59_338)
 3. Opción correcta (#answer-59_341)
 4. Incorrecto (#answer-59_344)
-

3.1.1.- Facetas de los tipos de datos.

¿Cuáles son las restricciones que podemos aplicar sobre los valores de los datos de un elemento o atributo?

Están definidos por las **facetas**, que solo pueden aplicarse sobre tipos simples utilizando el elemento **xs:restriction**. Se expresan como un elemento dentro de una restricción y se pueden combinar para lograr restringir más el valor del elemento. Son, entre otros:

- **length, minlength, maxlenght** : Longitud del tipo de datos.
- **enumeration** : Restringe a un determinado conjunto de valores.
- **whitespace** : Define el tratamiento de espacios (`preserve/replace, collapse`).
- **(max/min)(In/Ex)clusive** : Límites superiores/inferiores del tipo de datos. Cuando son Inclusive el valor que se determine es parte del conjunto de valores válidos para el dato, mientras que cuando se utiliza Exclusive, el valor dado no pertenece al conjunto de valores válidos.
- **totalDigits, fractionDigits** : número de dígitos totales y decimales de un número decimal.
- **pattern** : Permite construir máscaras que han de cumplir los datos de un elemento. La siguiente tabla muestra algunos de los caracteres que tienen un significado especial para la generación de las máscaras.

Elementos para hacer patrones.

Patrón	Significado
[A-Z a-z]	Letra.
[A-Z]	Letra mayúscula.
[a-z]	Letra minúscula.
[0-9]	Dígitos decimales.
\D	Cualquier carácter excepto un dígito decimal.
(A)	Cadena que coincide con A.
A B	Cadena que es igual a la cadena A o a la B.
AB	Cadena que es la concatenación de las cadenas A y B.
A?	Cero o una vez la cadena A.

A+	Una o más veces la cadena A.
A*	Cero o más veces la cadena A.
[abcd]	Alguno de los caracteres que están entre corchetes.
[^abcd]	Cualquier carácter que no esté entre corchetes.
\t	Tabulación.



Ejercicio Resuelto 1

1.- Creación de una cadena de texto con una longitud máxima de 9 caracteres y dos valores posibles.

Mostrar retroalimentación



Ejercicio Resuelto 2

2.- Creación de un elemento en el que se respetan los espacios tal y como se han introducido.

Mostrar retroalimentación



Ejercicio Resuelto 3

3.- Creación de un elemento calificaciones de dos dígitos cuyo valor es un número entero comprendido entre 1 y 10, ambos inclusive.

Mostrar retroalimentación



Ejercicio Resuelto 4

4.- Creación de la máscara de un DNI mediante `pattern` .

Mostrar retroalimentación

3.2.- Elementos del lenguaje.

A continuación se listan los más usados:

- ☐ Esquema, **xs:schema** , contiene la definición del esquema.
- ☐ Tipos complejos, **xs:complexType** , define tipos complejos.
- ☐ Tipos simples, **xs:simpleType** , permite definir un tipo simple restringiendo sus valores.
- ☐ Restricciones, **xs:restriction** , permite establecer una restricción sobre un elemento de tipo base.
- ☐ Agrupaciones, **xs:group** , permite nombrar agrupaciones de elementos y de atributos para hacer referencia a ellas.
- ☐ Secuencias, **xs:sequence** , permite construir elementos complejos mediante la enumeración de los que les forman.
- ☐ Alternativa, **xs:choice** , representa alternativas, hay que tener en cuenta que es una o-exclusiva.
- ☐ Contenido mixto, definido dando valor true al atributo **mixed** del elemento **xs:complexType** , permite mezclar texto con elementos.
- ☐ Secuencias no ordenadas, **xs:all** , representa a todos los elementos en cualquier orden.



Autoevaluación

Para hacer un elemento complejo formado por un listado de elementos en los que importa el orden hay que usar el elemento:

- ☐ `<xs:choice>`.
- ☐ `<xs:group>`.
- ☐ `<xs:all>`.
- ☐ `<xs:sequence>`.

No es correcta porque este elemento permite elegir entre elementos.

Incorrecta, porque este elemento permite hacer agrupaciones de elementos y atributos para hacer referencia a ellos dentro del esquema.

No es la respuesta correcta porque no respeta el orden de los elementos.

¡Genial! Ánimo, que está muy bien.

Solución

1. Incorrecto (#answer-60_144)
2. Incorrecto (#answer-60_347)
3. Incorrecto (#answer-60_350)
4. Opción correcta (#answer-60_353)



Debes conocer

Este enlace te permitirá consultar las estructuras del estándar XML Schema.

[XML Schema Estructuras](#)



Ejercicio Resuelto

Define un esquema correspondiente a un documento XML para estructurar la información personal sobre los alumnos de un centro educativo.

[Mostrar retroalimentación](#)

3.3.- Definición de tipos de datos XML Schema.

En los **DTD** se diferencia entre los elementos terminales y los no terminales ¿en este caso también?

Si, este lenguaje permite trabajar tanto con datos simples como con estructuras de datos complejos, es decir, compuestos por el anidamiento de otros datos simples o compuestos.



Q [Ministerio de Educación](#) (Uso Educativo-nc)

- ❑ **Tipos de datos simples.** Estos datos se suelen definir para hacer una restricción sobre un tipo de datos **XSD** ya definido y establece el rango de valores que puede tomar. También se pueden crear tipos de datos simples basados en listas de valores utilizando el atributo `derivedBy` de `simpleType`.
- ❑ **Tipos de datos compuestos.** El elemento `xsd:complexType` permite definir estructuras complejas de datos. Su contenido son las declaraciones de elementos y atributos, o referencias a elementos y atributos declarados de forma global. Para determinar el orden en que estos elementos aparecen en el documento **XML** se utiliza el propio elemento.



Ejercicio Resuelto 1

1.- Creación de un elemento simple de nombre `edad` que representa la edad de un alumno de la **ESO**, por tanto su rango está entre los 12 y los 18 años.

Mostrar retroalimentación



Ejercicio Resuelto 2

2.- Creación de una lista con los días de la semana en letras.

Mostrar retroalimentación



Ejercicio Resuelto 3

3. Creación de un elemento compuesto de nombre alumno, formado por los elementos nombre, apellidos y web personal (contiene el atributo href).

Mostrar retroalimentación

3.4.- Asociación con documentos XML.

Una vez que tenemos creado el **fichero XSD** ¿cómo lo **asociamos a un fichero XML**?

El modo de asociar un esquema a un documento **XML** es un espacio de nombres al ejemplar del documento, donde se indica la ruta de localización de los ficheros esquema mediante su **URI**, precedida del prefijo " **xsi:** ".



Ejercicio Resuelto

Dado el ejemplo de esquema de la sección 3.2. construye un documento **XML** que cumpla las especificaciones definidas en el archivo **XML Schema**.

Mostrar retroalimentación



Autoevaluación

La asociación de un documento XML a un esquema se hace en:

- ☐ El prólogo del documento XML.
- ☐ La definición de tipo de datos.
- ☐ El ejemplar.
- ☐ En una sección llamada declaración del esquema que se sitúa entre el prólogo y el ejemplar.

No es correcta porque el prólogo de un documento XML define la versión, el código y la independencia del documento para su interpretación.

Incorrecta, porque este apartado sirve para incorporar los DTD.

Muy bien, correcto.

Incorrecto, este apartado no existe.

Solución

1. Incorrecto (#answer-62_144)
2. Incorrecto (#answer-62_356)
3. Opción correcta (#answer-62_359)

4. Incorrecto (#answer-62_362)

3.5.- Documentación del esquema.

Una vez que hemos visto como crear un esquema vamos a ver el modo de incorporar cierta documentación (quién es el autor, limitaciones de derechos de autor, utilidad del esquema, etc.) al mismo.

Podemos pensar que un método para añadir esta información es utilizar comentarios. El problema es que los analizadores no garantizan que los comentarios no se modifiquen al procesar los documentos y por tanto, que los datos añadidos no se pierdan en algún proceso de transformación del documento.



Q

soyignatius - www.flickr.com
(CC BY-NC-ND)

En lugar de usar los comentarios, **XML Schema** tiene definido un elemento `xs:annotation` que permite **guardar información adicional**. Este elemento a su vez puede contener una combinación de otros dos que son:

- `xs:documentation`, además de contener elementos de esquema puede contener elementos **XML** bien estructurados.
También permite determinar el idioma del documento mediante el atributo `xml:lang`.
- `xs:appinfo`, se diferencia muy poco del elemento anterior, aunque lo que se pretendió inicialmente era que `xs:documentation` fuese legible para los usuarios y que `xs:appinfo` guardase información para los programas de software.
También es usado para generar una ayuda contextual para cada elemento declarado en el esquema.

EJEMPLO

Ejemplo de documentación de un esquema.

```
1  <xs:schema xmlns:xsi=http://www.w3.org/2001/XMLSchema>
2    <xs:annotation>
3      <xs:documentation xml:lang="es-es">
4        Materiales para formación e-Learning
5        <modulo>Lenguajes de marcas y sistemas de gestión de informa
6        <fecha_creación> 2011</fecha_creacion>
7        <autor> Nuky La Bruji</autor>
8      </xs:documentation>
9    </xs:annotation>
10   <xs:element name="lmsgi" type=xs:string>
11     <xs:annotation>
12       <xs:appinfo>
13         <texto_de_ayuda>Se debe de introducir el nombre completo
14       </xs:appinfo>
15     </xs:annotation>
16   </xs:element>
17 </xs:schema>
```



Autoevaluación

La mejor solución para documentar un esquema es usar los comentarios:

- ☐ Verdadero.
- ☐ Falso.

No es correcta. Aunque pueden usarse, no es la mejor opción porque no se puede garantizar que no se pierdan en alguna transformación del fichero.

Genial, correcto.

Solución

1. Incorrecto (#answer-64_144)
 2. Opción correcta (#answer-64_365)
-

4.- Herramientas de creación y validación.



Caso práctico

Antes de comenzar a trabajar con la normalización de los documentos que utiliza la empresa de **María y Félix. Marina** le presenta a **Juan** un informe sobre las diferentes herramientas que pueden facilitarles el trabajo de edición y validación de los documentos **XSD y XML**.

Juan hará un estudio de costes y escogerá alguna de ellas para realizar el trabajo.



Q [Jonny Goldstein](#)

(<https://www.flickr.com/photos/>

(CC BY)



Igual que hasta ahora, para crear y validar los documentos **XML** y los esquemas basta con un editor de texto plano y un navegador. ¿Pero no hay ninguna herramienta que nos facilite el trabajo? Pues sí, existen aplicaciones que permiten al usuario visualizar, validar y editar documentos en el lenguaje **XML**. Algunos de estos productos son:

- ❑ [Editix XML Editor](#) (Versiones open source y comercial).
- ❑ [XMLFox](#) (freeware).
- ❑ [Altova XML Spy Edición Estándar](#) (comercial).
- ❑ Editor XML [xmlBlueprint](#). (comercial)
- ❑ [Stylus Studio 2001](#) (comercial).
- ❑ [Oxygen XML Editor](#) (comercial).
- ❑ [Exchanger XML Editor](#) (comercial)
- ❑ [XML copy editor](#) (open source).



[xornalcerto - www.flickr.com](#)

(CC BY)

5.- Recursos adicionales.



Para saber más

Información adicional:

- Fundamentos de XML Schema (<https://www.w3.org/TR/xmlschema-0/>).
 - Estructuras de XML Schema (<https://www.w3.org/TR/xmlschema-1/>).
 - w3schools - Tutorial XML Schema
(https://www.w3schools.com/xml/schema_intro.asp).
 - w3schools - Tutorial DTD
(https://www.w3schools.com/xml/xml_dtd_intro.asp)
(<https://sites.google.com/site/todoxmldtd/home>).
 - Abrirllave.com Tutorial de XML Schema
(<https://www.abrirllave.com/xsd/>).
-

